1. Import Libraries and Generate Synthetic Data

We will import the required libraries and generate a synthetic 2D dataset to easily visualize the decision boundaries of different classifiers.

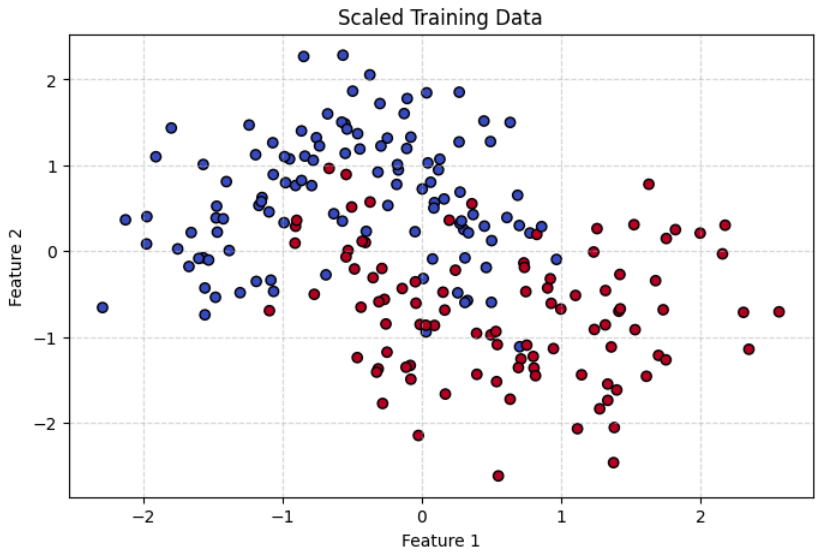
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Generate a synthetic 2D dataset (two interleaving half circles)
X, y = make_moons(n_samples=300, noise=0.3, random_state=42)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Scale the features (highly recommended for SVM and Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Plot the dataset
plt.figure(figsize=(8, 5))
plt.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1], c=y_train, cmap='coolwarm', edgecolors='k')
plt.title("Scaled Training Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



2. Train and Evaluate SVM vs. Other Models

We will train three models:

1. **Support Vector Machine (SVM)** with an RBF kernel.
2. **Logistic Regression** (a linear classifier baseline).
3. **Random Forest** (an ensemble-based non-linear classifier).

```
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Initialize models
models = {
    "Support Vector Machine (RBF Kernel)": SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42),
    "Logistic Regression": LogisticRegression(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)
}

# Train models and display performance metrics
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    print(f"=== {name} ===")
    print(f"Accuracy: {acc:.4f}")
    print(classification_report(y_test, y_pred))
    print("-" * 40)
```

```
=== Support Vector Machine (RBF Kernel) ===
Accuracy: 0.8778
              precision    recall  f1-score   support

           0       0.86      0.84      0.85        38
           1       0.89      0.90      0.90        52

    accuracy                           0.88        90
   macro avg       0.88      0.87      0.87        90
weighted avg       0.88      0.88      0.88        90

----------------------------------------
=== Logistic Regression ===
Accuracy: 0.8778
              precision    recall  f1-score   support

           0       0.85      0.87      0.86        38
           1       0.90      0.88      0.89        52

    accuracy                           0.88        90
   macro avg       0.87      0.88      0.88        90
weighted avg       0.88      0.88      0.88        90

----------------------------------------
=== Random Forest ===
Accuracy: 0.8778
              precision    recall  f1-score   support

           0       0.89      0.82      0.85        38
           1       0.87      0.92      0.90        52

    accuracy                           0.88        90
   macro avg       0.88      0.87      0.87        90
weighted avg       0.88      0.88      0.88        90

----------------------------------------
```

3. Visualize Decision Boundaries

Let's visualize and compare how each model partitions the feature space to make decisions.

```
# Define grid for plotting decision boundaries
x_min, x_max = X_train_scaled[:, 0].min() - 0.5, X_train_scaled[:, 0].max() + 0.5
y_min, y_max = X_train_scaled[:, 1].min() - 0.5, X_train_scaled[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02))

fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

for ax, (name, model) in zip(axes, models.items()):
    # Predict boundaries
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot boundary contours
    ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

    # Scatter plot of training data points
    scatter = ax.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1], c=y_train, cmap='coolwarm', edgecolors='k')

    ax.set_title(name)
    ax.set_xlabel("Feature 1")
    if ax == axes[0]:
        ax.set_ylabel("Feature 2")
    ax.grid(True, linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```

